

raking: Design Document

Version 3 (revised to match the implementation)

2026-06-01

1 Overview

raking is an R package for representative sample weighting: finding weights that make a survey sample match known population targets, following the method of Barratt, Angeris & Boyd (2021), “Optimal Representative Sample Weighting.”

You build a weighting model by piping a few functions together: pick a global loss, add categorical margin targets and continuous mean calibrations, set a penalty and weight bounds. The model carries no data. You then `fit()` it to a dataset to get weights, and you can fit the same model to several datasets.

raking shares the ADMM solver math with the existing **rsw** (Python reference) and **regrake** (R, formula-driven) packages, and reproduces their results, but is an independent, R-native implementation with its own interface.

This version (v3) supersedes the v2 draft. Section 11 lists the substantive changes; the most important is that the meaning of `tol` and `loss` has been made precise (Sections 4.2, 6.2).

2 Scope

In scope:

- The representative sample weighting problem: weights w minimizing a loss against targets plus a penalty on the weights, subject to weight bounds.
- Categorical margin targets and continuous mean calibration constraints, each either exact or matched within a tolerance band.
- Base (design) weights.

Out of scope (methods in the paper that are not part of **raking**):

- Representative selection (choosing k equal-weight samples).
- Resampling from the fitted weights.

Planned, not yet implemented: `calibrate(var(x) ~ v)` and `calibrate(quantile(x, p) ~ v)`. The first version supports `mean()`.

3 Mathematical framework

`raking` solves the representative sample weighting problem from Barratt et al. (2021), eq. (2), on the sum-to- n weight scale used throughout this package:

$$\begin{aligned} & \text{minimize} && \ell(f, f_{\text{des}}) + \lambda r(w) \\ & \text{subject to} && f = F(w/n) \\ & && w \geq 0, \quad \mathbf{1}^T w = n, \quad w_{\min} \leq w \leq w_{\max} \end{aligned}$$

with variables $w \in \mathbb{R}^n$ and $f \in \mathbb{R}^m$, and penalty multiplier $\lambda \geq 0$.

- w is the weight vector, one entry per row of the data. Weights sum to n , so a weight of 1 leaves a row at its natural influence, 2 doubles it, 0.5 halves it.
- f is the vector of achieved quantities (weighted proportions and means), linked to w through $f = F(w/n)$.
- F is the $m \times n$ feature matrix, assembled at `fit()` time. Row j of $F(w/n)$ is a weighted proportion (for a margin level) or a weighted mean (for a calibration).
- f_{des} are the target values.
- ℓ is the loss measuring discrepancy between achieved and target values. It may be infinite-valued to express hard constraints on f (see 6.2).
- $r(w)$ is the penalty on the weights (e.g. closeness to uniform).
- $w_{\min}, w_{\max}$ are element-wise weight bounds, enforced as hard constraints by projection at every ADMM iteration.

Relationship to the paper. The paper uses $\mathbf{1}^T w = 1$ (probability simplex). This package uses $\mathbf{1}^T w = n$ for the user-facing scale. The solver operates internally on the $\mathbf{1}^T w = 1$ scale and rescales output weights to $\mathbf{1}^T w = n$ before returning them.

4 User-facing API

4.1 Building a model

```
model <- raking(loss = "KL", name = "wt") |>
  target(age_targets) |> # exact margin
  target(region_targets, tol = 0.02) |> # penalized band, +/- 0.02
  calibrate(mean(income) ~ 58000) |> # exact mean
  penalty(entropy(), lambda = 1) |>
  bounds(min = 0.5, max = 2)
```

`raking(loss, name)` creates a weighting model (an S7 object of class `raking_model`).

- `loss` is one of "KL", "L1", "L2", "chi_square", default "KL". It selects the soft discrepancy measure used inside a tolerance band (see 4.2 and 6.2). It has no effect on a constraint matched exactly (`tol = 0`).
- `name` is the weight-column name for downstream use. Default "wt".

The remaining functions are pipe steps that each take a model and return an updated model. Data-free validation (loss is a known key, `tol >= 0`, bound ordering) happens immediately; data-dependent validation happens at `fit()`.

4.2 `target()` — categorical margin constraints

```
target(model, df, tol = 0)
```

`df` is a data frame with one or more categorical variable columns (character or factor) and exactly one column named `p` (proportions, must sum to 1) or `n` (counts, normalized internally). One variable column gives a simple margin; two or more give a joint margin (one constraint row per combination present in `df`).

`tol` sets how strictly this margin is matched:

- `tol = 0` (default): **exact** matching. The achieved proportions equal the targets. Implemented as a hard equality constraint, so the global `loss` is irrelevant for this margin.
- `tol > 0`: a **penalized band**. The global `loss` pulls the achieved proportions toward the exact targets, while a hard wall keeps every level within $\pm \text{tol}$ of its target. So the band is a guarantee and the loss is a preference inside it. As `tol` grows the constraint approaches pure soft matching; at `tol = 0` the band collapses to the target point (exact).

Note: this is a *penalized* band, distinct from an *indifferent* band (where any point inside scores equally, as in `regrake`'s `rr_range`). See 6.2.

4.3 `calibrate()` — continuous mean constraints

```
calibrate(model, formula, tol = 0)
```

The formula left-hand side is `mean(var)`; the right-hand side is the target value. `tol` (in the variable's own units) behaves exactly as for `target()`: `tol = 0` matches the weighted mean exactly, `tol > 0` is a penalized band of half-width `tol` around the target. The left-hand side is captured unevaluated and resolved at `fit()` time.

4.4 `penalty()` — weight penalty

```
penalty(model, reg, lambda = 1)
```

Sets the penalty $r(w)$ and multiplier `lambda`. `reg` is one of:

- `entropy()` — KL divergence of w from the base weights (uniform by default). The standard choice; pulls weights toward the base.
- `kl(prior)` — KL divergence from an explicit positive reference vector.
- `sum_squares()` — sum of squared deviations from the base weights.

With no `penalty()` step the model is unpenalized ($\lambda = 0$). A non-uniform base (supplied to `fit()`) turns the default/entropy penalty into a KL penalty toward the base distribution, so a supplied base is never silently ignored. At most one `penalty()` call per model; a second replaces the first with a message.

4.5 `bounds()` — weight bounds

```
bounds(model, min = 0, max = Inf)
```

Element-wise bounds on individual weights (on the sum- n scale), enforced as hard constraints by projection onto the bounded probability simplex at every ADMM iteration. Validated at `bounds()` time: $\min \in [0, 1)$, $\max > 1$, $\min < \max$. Since the mean weight is 1, $\min \leq 1 \leq \max$ is required for

feasibility. Default `min = 0`, `max = Inf` leaves only $w \geq 0$. At most one `bounds()` call; a second replaces the first with a message.

Feasibility caveat: a hard margin (`tol = 0`) can conflict with tight bounds (for example, `target(M = 0.5)` on 4 M and 1 F needs each M weight = 0.625, which is infeasible under `min = 0.9`). The fit then does not converge and returns the best iterate with a warning (see 8).

4.6 `fit()` — attach data and solve

```
fit(model, data, base = NULL, control = rake_control(), ...)
```

Attaches data, assembles F and f_{des} , validates, runs the ADMM solver, and returns a `raking_fit`.

- `data` is a data frame.
- `base` is base/design weights: a column name in `data`, or a numeric vector of length `nrow(data)`, all strictly positive. Default uniform. Internally rescaled to the probability simplex; output weights are on the sum- n scale.
- `control` is a `rake_control()` (settings may also be passed via `...` and are merged into the defaults).
- `fit()` dispatches on `raking_model` via the generic from the `generics` package.

The same model may be fit to multiple datasets independently.

4.7 `rake_control()` — solver settings

```
rake_control(maxiter = 5000, rho = 50, margin_tol = 1e-4,
             eps_abs = 1e-5, eps_rel = 1e-5, verbose = FALSE)
```

Parameter	Description
<code>maxiter</code>	Maximum ADMM iterations
<code>rho</code>	ADMM penalty parameter (default 50, matching <code>regrake</code>)
<code>margin_tol</code>	Margin-based convergence tolerance (default <code>1e-4</code>), converted internally to $\varepsilon = \text{margin_tol} / \sqrt{m + 2n}$. Set to <code>NULL</code> to use raw <code>eps_abs/eps_rel</code> .
<code>eps_abs, eps_rel</code>	Raw ADMM residual tolerances (used when <code>margin_tol = NULL</code>)
<code>verbose</code>	Print residuals periodically

Supplying `eps_abs/eps_rel` without `margin_tol` sets `margin_tol = NULL` automatically.

4.8 Extracting results

Function	Returns
<code>weights(fit)</code>	Numeric vector, length n , sum = n , input row order

Function	Returns
<code>summary(fit)</code>	Prints the balance table + Kish ESS/deff + convergence; returns the fit invisibly
<code>print(fit)</code>	Short summary: loss, n , convergence status, weight range, Kish ESS

5 The weighting model object

`raking()` returns an S7 object of class `raking_model` with declared, typed properties and a validator that runs on construction and modification.

Property	S7 type	Description
<code>loss</code>	<code>class_character</code>	One of "KL", "L1", "L2", "chi_square"
<code>name</code>	<code>class_character</code>	Output weight column name
<code>targets</code>	<code>class_list</code>	Margin specs; each is <code>list(df, tol)</code>
<code>calibrations</code>	<code>class_list</code>	Specs; each is <code>list(formula, tol)</code>
<code>penalty</code>	<code>class_any</code>	An S7 penalty object, or NULL
<code>lambda</code>	<code>class_numeric</code>	Penalty multiplier
<code>bounds</code>	<code>class_any</code>	<code>list(min, max)</code> or NULL

The model holds no data. Property access uses `@`.

6 Internals

6.1 Building F

At `fit()` time `raking` assembles F ($m \times n$) and f_{des} :

- Each `target()` margin contributes one row per level (per cell for a joint margin): the 0/1 indicator of membership. The achieved quantity is the weighted proportion of that level.
- Each `calibrate()` constraint contributes one row of the raw variable values. The achieved quantity is the weighted mean.

Normalization. Calibration rows are divided by their target value before solving (so the normalized target is 1), bringing proportion and mean rows to a comparable scale. Achieved values and tolerances are de-normalized before reporting.

6.2 Losses and the meaning of `tol`

The `loss` key selects the soft discrepancy ℓ used inside a band:

Key	Soft discrepancy	Proximal operator
"KL"	$\frac{1}{2} \sum_j f_j \log(f_j / f_{\text{des},j})$	closed form via Lambert W
"L2"	$\ f - f_{\text{des}}\ _2^2$	closed form
"L1"	$\ f - f_{\text{des}}\ _1$	soft thresholding

Key	Soft discrepancy	Proximal operator
"chi_square"	$\sum_j (f_j - f_{\text{des},j})^2 / f_{\text{des},j}$	closed form (weighted L2)

The KL discrepancy carries a factor $\frac{1}{2}$, matching **rs**w and **regrake**; this is required to reproduce their results. **chi_square** is implemented as a weighted least squares with per-element weight $1/\sqrt{f_{\text{des}}}$, which is algebraically Pearson χ^2 ; it requires positive targets.

How a constraint with tolerance t is built:

- $t = 0$: a hard **equality** loss. $\ell(f) = 0$ if $f = f_{\text{des}}$, else $+\infty$. Its prox returns the target. The global **loss** is irrelevant.
- $t > 0$: a **penalized band**. The achieved value is pulled toward the target by the soft loss above, then clipped into the hard band $[f_{\text{des}} - t, f_{\text{des}} + t]$. Concretely the prox is $\text{clip}(\text{prox}_{\text{soft}}(v), f_{\text{des}} - t, f_{\text{des}} + t)$. Because the soft losses are separable and the band is a box, this clipped soft prox is the exact proximal operator of “soft loss subject to the box.”

This penalized band differs from an *indifferent* band (**regrake**’s **rr_range**), where any point inside the band scores equally. Here the loss prefers the exact target inside the band; the band is only an outer wall. A consequence: a **tol** larger than the point where the band stops binding gives pure soft matching, not an indifferent region.

6.3 Penalties

Constructor	$r(w)$	Proximal operator
entropy()	$\sum_i w_i \log(w_i/b_i)$	closed form via Lambert W
kl(prior)	$\sum_i w_i \log(w_i/p_i)$	closed form via Lambert W
sum_squares()	$\sum_i (w_i - b_i)^2$	closed form (linear)

b_i is the base weight for row i (uniform $1/n$ by default on the simplex scale). **entropy()** with default base is maximum-entropy regularization; with a design base it pulls toward those. The penalty classes are small internal S7 objects (exported so they can be unit-tested and constructed directly).

6.4 The solver

raking uses the ADMM algorithm from the paper, on the $\mathbf{1}^T w = 1$ scale. Following eq. (4) it introduces three consensus copies of w : a penalty copy $\tilde{w}$, a projection copy $\bar{w}$, and a loss copy w . The w -update solves a linear system with coefficient matrix

$$Q = \begin{bmatrix} 2I & F^T \\ F & -I \end{bmatrix},$$

which is quasi-definite. R lacks a specialized quasi-definite solver, so the implementation adds a tiny diagonal regularizer ($Q + \text{damp} \cdot I$, with $\text{damp} \approx \max(10^{-8}, 10^{-6} \cdot \max_i \sum_j |Q_{ij}|)$) and factorizes once with **Matrix::Cholesky** (sparse LDL^T), reused every iteration. This damping bounds the achievable accuracy at roughly 2×10^{-4} regardless of the convergence tolerance; **rs**w reaches tighter accuracy using QDLDL.

Each iteration applies the loss prox (per block), the penalty prox, and the projection Π onto $\{w \geq 0, \mathbf{1}^T w = 1, w_{\min}/n \leq w \leq w_{\max}/n\}$ (Duchi et al. 2008, bounded simplex), then a back-solve and dual updates. Because the bound set is part of Π , bounds are hard constraints, satisfied at every iterate.

Returned weights. The solver returns the final feasible projection iterate $\bar{w}$ (rescaled to sum n), consistent with `rs` and `regrake`. The paper’s “best-iterate” tracking applies to the out-of-scope representative-selection (Boolean) path and is not used here. For penalized-band constraints the prox clips the loss copy into the band every iteration, so a residual-converged fit satisfies the band up to the convergence tolerance; no separate band check is needed.

Convergence is declared when the ADMM primal and dual residuals fall below ε . If `maxiter` is reached first, a warning is issued and the best iterate is returned with `convergence$converged = FALSE`.

6.5 tol and margin_tol

`tol` (per constraint) changes the optimization problem: 0 is exact, $t > 0$ is a penalized band. `margin_tol` (global, in `rake_control()`) only controls when ADMM stops; it is converted to a size-scaled residual threshold and does not change what is solved.

7 The fitted model

`fit()` returns an S7 object of class `raking_fit`:

Property	S7 type	Description
<code>weights</code>	<code>class_numeric</code>	Weight vector, length n , sum = n
<code>balance</code>	<code>class_data.frame</code>	One row per constraint level: <code>variable</code> , <code>level</code> , <code>target</code> , <code>achieved</code> , <code>difference</code> , <code>satisfied</code>
<code>convergence</code>	<code>class_list</code>	<code>converged</code> , <code>iterations</code> , <code>primal_residual</code> , <code>dual_residual</code>
<code>diagnostics</code>	<code>class_list</code>	<code>weight</code> and <code>margin</code> diagnostics (below)
<code>model</code>	<code>raking_model</code>	The model used
<code>call</code>	<code>class_any</code>	Captured <code>fit()</code> call
<code>name</code>	<code>class_character</code>	Weight column name

Diagnostics: `weight_range`, `weight_mean`, `weight_sd`, `kish_deff` = $1 + \text{var}(w)/\text{mean}(w)^2$, `kish_ess` = $n/\text{kish_deff}$, `max_abs_diff`, `max_pct_diff`.

8 Validation and errors

At `fit()` time, `raking` errors clearly on: a target variable absent from the data; a data level with no corresponding target (or a target level absent from the data, or overlapping target cells); malformed

n/p , or p not summing to 1 within a small tolerance; NAs in a variable used by a constraint; a non-zero calibration target requirement. Bounds infeasibility at the model level ($\min \geq 1$ or $\max \leq 1$) is caught eagerly at `bounds()` time. A fit that does not converge within `maxiter` is a warning (not an error): it returns the best iterate with `convergence$converged = FALSE` and full diagnostics. Model-building functions validate their own arguments eagerly.

9 Package layout

S7 throughout. User-visible classes `raking_model` and `raking_fit`, plus small internal S7 loss and regularizer classes.

File	Contents
<code>model.R</code>	<code>raking()</code> , <code>raking_model</code> class, print method
<code>pipe.R</code>	<code>target()</code> , <code>calibrate()</code> , <code>penalty()</code> , <code>bounds()</code>
<code>losses.R</code>	loss classes (<code>EqualityLoss</code> , <code>LeastSquaresLoss</code> , <code>InequalityLoss</code> , <code>KLLoss</code> , <code>L1Loss</code> , <code>BandedLoss</code>) and their proxes
<code>regularizers.R</code>	regularizer classes and proxes
<code>penalty.R</code>	<code>entropy()</code> , <code>kl()</code> , <code>sum_squares()</code> constructors
<code>fit.R</code>	<code>fit()</code> method, F assembly, normalization, validation, <code>raking_fit</code>
<code>solver.R</code>	<code>admm()</code> , simplex and bounded-simplex projection
<code>result.R</code>	<code>weights()</code> , <code>summary()</code> , Kish helpers
<code>control.R</code>	<code>rake_control()</code> , <code>margin_tol</code> scaling
<code>generics.R</code>	<code>prox</code> , <code>evaluate</code> S7 generics
<code>raking-package.R</code>	package doc, <code>.onLoad</code> S7 method registration

Dependencies: `S7` (objects), `Matrix` (sparse storage and LDL^T), `lamW` (Lambert W for KL/entropy proxes), `generics` (the `fit` generic). Suggested (testing only): `testthat`, `regrake`.

10 Testing

The test suite covers, with reference values generated from a faithful port of the `rsW` solver and cross-checked against `rsW` where possible:

- **Proximal operators.** Each loss and penalty prox against a reference value.
- **End-to-end correctness.** Margins and means matched exactly; weights sum to n ; weights in input row order; positive.
- **Known analytic case.** Post-stratification (one binary variable, L2, no penalty) against the closed form, to 10^{-3} (the $\sim 2 \times 10^{-4}$ damping floor prevents the paper's 10^{-6}).
- **Penalized band.** `tol > 0` pulls the achieved value toward the target; `loss` changes the result inside a non-binding band; `tol = 0` is exact and `loss`-independent.

- **Kish ESS/deff** against the manual formulas.
- **Bounds are hard.** Every weight within bounds, including a binding case.
- **Cross-package parity.** Against **regrake** on exact constraints to 10^{-4} . (Range constraints differ by design: penalized vs. indifferent band.)
- **S7 validation.** Invalid loss, negative tol, out-of-range bounds, bad regularizer parameters error at the point of construction.
- **Error cases.** Absent variable, level mismatch, NAs, malformed **p**, infeasible bounds, non-convergence.
- **Edge cases.** Single-row data, single-level margins, factor variables, count normalization, multiple margins and calibrations, model reuse across datasets, unused/NA non-constraint columns, base as column vs. vector, replaced penalty and bounds, non-binding bands.

11 Changes from v2

1. **tol is precise now.** v2 said `tol = 0` meant “exact matching” while also saying the global `loss` chose the discrepancy; those are contradictory. v3: `tol = 0` is a hard equality (loss irrelevant); `tol > 0` is a penalized band built from the soft `loss` clipped into $[f_{\text{des}} \pm \text{tol}]$.
2. **calibrate() gains tol.** Means can be matched within a band, not only exactly.
3. **Penalized vs. indifferent band.** v3 documents that the band pulls toward the target (unlike **regrake**’s indifferent `rr_range`), and that they agree on exact constraints but can differ on ranges.
4. **KL scale.** Documented as $\frac{1}{2}$ for **rsw/regrake** parity.
5. **Returned iterate.** Documented that **raking** returns the final feasible iterate $\bar{w}$; best-iterate tracking is the out-of-scope Boolean path.
6. **Accuracy floor.** Documented the $\sim 2 \times 10^{-4}$ damping floor and the 10^{-6} analytic target being unreachable with the R solver.
7. **Dependencies.** **rlang** removed (base R formula handling is used). Loss and penalty classes are exported (for testing and direct construction).
8. **raking_fit.** Full property set specified (`balance`, `convergence`, `diagnostics`, `model`, `call`).

12 References

Barratt, Angeris & Boyd (2021). *Optimal Representative Sample Weighting*. SSRN 3604610. Boyd et al. (2010). *Distributed Optimization and Statistical Learning via ADMM*. Duchi et al. (2008). *Efficient Projections onto the ℓ_1 -Ball*. ICML.